Alpaca benchmark configs

Fourteen ready-to-copy configs for `alpaca benchmark`, one per common setup: OasisLMF releases, OasisLMF git branches, GitHub and S3 models, tests, shared instances and stored S3 baselines. Each one shows what it launches before you spend anything, and the end of the guide shows what happens when a run fails.

Before you run · How it works · OasisLMF releases · OasisLMF git branches · Model locations and tests · Stored S3 baselines · Everything else · Shared instances · When a run fails · Rejected configs

Before you run one

Every file uses placeholders for the account-specific values. Replace them with your own.

KEY	PLACEHOLDER	REPLACE WITH
`SECURITY_GROUP_ID`	`sg-xxxxxxxxxxxxxxxx`	A security group in your VPC
`SUBNET_ID`	`subnet-xxxxxxxx`	A subnet in the same VPC
`IAM_INSTANCE_PROFILE`	`YourSSMInstanceProfile`	An instance profile with SSM access (`AmazonSSMManagedInstanceCore`) and, for S3 models or baselines, S3 read access
`AWS_PROFILE`	`oasis`	Your named AWS CLI profile, or remove the key to use the default credentials
`REPO_LOCATIONS`	`s3://my-model-bucket`	Your model's bucket (the PiWind GitHub URL works as is)
`BENCHMARK_BUCKET`	`s3://my-benchmark-bucket`	The bucket your baselines are stored in

Results go to the `RESULT_DIRECTORY` each file names (e.g. `./runs_compare_versions`), relative to where you run `alpaca`. Each run gets its own subfolder, next to a combined `benchmark_report.txt`.

How a benchmark is built

`model location` × `test` × `OasisLMF install` = **one run each**

- Each run is an ordinary `alpaca model` run on its own throwaway EC2 instance, up to 8 at a time. With `TESTS_PER_INSTANCE` set to `shared`, a model's tests at one version take turns on a single instance instead.
- The **OasisLMF install** comes from `OASISLMF_VERSIONS` (PyPI releases), `OASISLMF_BRANCHES` (git branches, installed from source) or the `OASISLMF_BASELINE_VERSION` / `OASISLMF_TEST_VERSION` pair. You need at least one.
- **What runs** is either `PATH_TO_OASISLMF_JSON` (one `oasislmf.json` per model) or `TESTS` (`tests/<name>/oasislmf.json` per test). Set exactly one.
- **Comparison** happens within each model + test group. The reference is the baseline when the baseline/test pair is set, and otherwise the fastest run. The rest are timed and have their output diffed against it, and different models or tests are never compared with each other.
- **Like for like:** every run saves a `manifest.json` with its instance type, AMI and checksums of its `oasislmf.json`, analysis settings and model settings. The report flags any run whose manifest differs from the reference's, such as changed model settings.
- With `BENCHMARK_BUCKET` set, a version already stored there is downloaded instead of run, at no EC2 cost. Branches are never stored, so they always run.
- **Failures don't stop the benchmark.** A failed run gets a `failure.txt` explaining why, the report lists it, and `alpaca benchmark` exits with status 1 (see When a run fails).

`2.5.8` runs on EC2 `2.5.7` downloaded from S3 `2.5.8 → S3` runs, then published to S3

OasisLMF releases

Released versions from PyPI, listed in `OASISLMF_VERSIONS`.

01 One timed run

1 instance

`01_single_version.json`

PiWind on OasisLMF 2.5.8. There's nothing to compare against, so you get step timings only.

PiWind	2.5.8


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "https://github.com/OasisLMF/OasisPiWind"
    ],
    "PATH_TO_OASISLMF_JSON": "./oasislmf.json",
    "OASISLMF_VERSIONS": [
        "2.5.8"
    ],
    "RESULT_DIRECTORY": "./runs_single_version"
}
```


02 Compare two releases

2 instances

`02_compare_versions.json`

PiWind on 2.5.7 and 2.5.8, with timings and an output diff. The usual "did the new release change anything?" check.

PiWind	2.5.7	2.5.8


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "https://github.com/OasisLMF/OasisPiWind"
    ],
    "PATH_TO_OASISLMF_JSON": "./oasislmf.json",
    "OASISLMF_VERSIONS": [
        "2.5.7",
        "2.5.8"
    ],
    "RESULT_DIRECTORY": "./runs_compare_versions"
}
```

OasisLMF git branches

Branches of the OasisLMF repo, installed from source and listed in `OASISLMF_BRANCHES`.

03 Compare two branches

2 instances

`03_compare_branches.json`

PiWind on OasisLMF `main` and `stable/2.5.x`, both installed from source.

PiWind	main	stable/2.5.x

> Use a branch name exactly as it appears in the OasisLMF repo, e.g. `feature/my-change`.


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "https://github.com/OasisLMF/OasisPiWind"
    ],
    "PATH_TO_OASISLMF_JSON": "./oasislmf.json",
    "OASISLMF_BRANCHES": [
        "main",
        "stable/2.5.x"
    ],
    "RESULT_DIRECTORY": "./runs_compare_branches"
}
```


04 Release against a branch

2 instances

`04_version_vs_branch.json`

2.5.8 from PyPI against OasisLMF `main`, to see whether unreleased work changes results or speed.

PiWind	2.5.8	main


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "https://github.com/OasisLMF/OasisPiWind"
    ],
    "PATH_TO_OASISLMF_JSON": "./oasislmf.json",
    "OASISLMF_VERSIONS": [
        "2.5.8"
    ],
    "OASISLMF_BRANCHES": [
        "main"
    ],
    "RESULT_DIRECTORY": "./runs_version_vs_branch"
}
```

Model locations and tests

GitHub or S3 models in `REPO_LOCATIONS`, and per-test runs with `TESTS`.

05 Private model from S3

2 instances

`05_s3_model.json`

The model is pulled from S3 instead of GitHub. The instance runs `aws s3 cp`, so `IAM_INSTANCE_PROFILE` needs read access to that bucket.

my-model-bucket	2.5.7	2.5.8


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket"
    ],
    "PATH_TO_OASISLMF_JSON": "./oasislmf.json",
    "OASISLMF_VERSIONS": [
        "2.5.7",
        "2.5.8"
    ],
    "RESULT_DIRECTORY": "./runs_s3_model"
}
```


06 Several models at once

4 instances

`06_multiple_models.json`

PiWind (GitHub) and an S3 model, each at 2.5.7 and 2.5.8. Each model is only compared with itself, and the report has one section per model.

PiWind	2.5.7	2.5.8
my-model-bucket	2.5.7	2.5.8


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "https://github.com/OasisLMF/OasisPiWind",
        "s3://my-model-bucket"
    ],
    "PATH_TO_OASISLMF_JSON": "./oasislmf.json",
    "OASISLMF_VERSIONS": [
        "2.5.7",
        "2.5.8"
    ],
    "RESULT_DIRECTORY": "./runs_multiple_models"
}
```

07 Several tests of one model

4 instances

07_tests.json

Two tests (e.g. different event sets) at two versions. test_1 is only compared with test_1, and test_2 with test_2.

test_1	2.5.7	2.5.8
test_2	2.5.7	2.5.8

```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket"
    ],
    "TESTS": [
        "test_1",
        "test_2"
    ],
    "OASISLMF_VERSIONS": [
        "2.5.7",
        "2.5.8"
    ],
    "RESULT_DIRECTORY": "./runs_tests"
}
```

Stored S3 baselines

Publishing to `BENCHMARK_BUCKET`, and reusing what's stored there.

08 Publish a baseline

1 instance

`08_publish_baseline.json`

Runs PiWind on 2.5.8 and stores the result in `BENCHMARK_BUCKET` as the 2.5.8 baseline. Any baseline already stored there is moved into its `history/` folder first, not overwritten.

PiWind	2.5.8 → S3


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "https://github.com/OasisLMF/OasisPiWind"
    ],
    "PATH_TO_OASISLMF_JSON": "./oasislmf.json",
    "OASISLMF_VERSIONS": [
        "2.5.8"
    ],
    "BENCHMARK_BUCKET": "s3://my-benchmark-bucket",
    "PUBLISH_BASELINE": true,
    "RESULT_DIRECTORY": "./runs_publish_baseline"
}
```


09 Publish a baseline per test

2 instances

`09_publish_baseline_tests.json`

Stores a 2.5.8 baseline for each test, under `my-model-bucket/test_1/2.5.8/` and `…/test_2/2.5.8/`.

test_1	2.5.8 → S3
test_2	2.5.8 → S3


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket"
    ],
    "TESTS": [
        "test_1",
        "test_2"
    ],
    "OASISLMF_VERSIONS": [
        "2.5.8"
    ],
    "BENCHMARK_BUCKET": "s3://my-benchmark-bucket",
    "PUBLISH_BASELINE": true,
    "RESULT_DIRECTORY":
"./runs_publish_baseline_tests"
}
```

10 Reuse whatever is stored

0–4 instances

`10_reuse_stored_baselines.json`

2.5.7 against 2.5.8 with `OASISLMF_VERSIONS`. Any version already in the bucket is downloaded instead of run.

test_1	2.5.7	2.5.8
test_2	2.5.7	2.5.8

> Shown with both versions already stored, so **nothing runs**. A version missing from the bucket would run live instead.


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket"
    ],
    "TESTS": [
        "test_1",
        "test_2"
    ],
    "OASISLMF_VERSIONS": [
        "2.5.7",
        "2.5.8"
    ],
    "BENCHMARK_BUCKET": "s3://my-benchmark-bucket",
    "RESULT_DIRECTORY":
"./runs_reuse_stored_baselines"
}
```


11 Always run the test version

2–4 instances

`11_baseline_vs_test_version.json`

The same comparison with the baseline/test pair. 2.5.7 comes from the bucket if it's stored there, and 2.5.8 always runs, even if it's stored too. 2.5.7 is the reference, so the report shows 2.5.8's change from it, e.g. `-38.8% vs baseline`.

test_1	2.5.7	2.5.8
test_2	2.5.7	2.5.8

> Use this to re-test a version after a model data or settings change, even though a baseline for it exists.


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket"
    ],
    "TESTS": [
        "test_1",
        "test_2"
    ],
    "OASISLMF_BASELINE_VERSION": "2.5.7",
    "OASISLMF_TEST_VERSION": "2.5.8",
    "BENCHMARK_BUCKET": "s3://my-benchmark-bucket",
    "RESULT_DIRECTORY":
"./runs_baseline_vs_test_version"
}
```

12 Pair plus a branch

2–3 instances

`12_baseline_vs_test_version_and_branch.json`

The baseline comes from S3, while the test version and OasisLMF `main` both run live and are compared against it.

test_1	2.5.7	2.5.8	main


```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket"
    ],
    "TESTS": [
        "test_1"
    ],
    "OASISLMF_BASELINE_VERSION": "2.5.7",
    "OASISLMF_TEST_VERSION": "2.5.8",
    "OASISLMF_BRANCHES": [
        "main"
    ],
    "BENCHMARK_BUCKET": "s3://my-benchmark-bucket",
    "RESULT_DIRECTORY":
"./runs_baseline_vs_test_version_and_branch"
}
```

Everything else

The less common keys.

13 Every tunable option

2–4 instances

`13_all_options.json`

Two S3 models with the pair, run one at a time, with a wider comparison tolerance, a bigger disk and a longer lifetime tag.

`my-model-bucket`	`2.5.7`	`2.5.8`
`my-other-model-bucket`	`2.5.7`	`2.5.8`

```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket",
        "s3://my-other-model-bucket"
    ],
    "TESTS": [
        "test_1"
    ],
    "OASISLMF_BASELINE_VERSION": "2.5.7",
    "OASISLMF_TEST_VERSION": "2.5.8",
    "BENCHMARK_BUCKET": "s3://my-benchmark-bucket",
    "EXECUTION_MODE": "sequential",
    "COMPARISON_TOLERANCE": 0.01,
    "DISK_GB": 200,
    "MAX_LIFETIME_HOURS": 4,
    "SSH_MAX_RETRIES": 100,
    "LOG_LEVEL": "INFO",
    "RESULT_DIRECTORY": "./runs_all_options"
}
```

Shared instances

Several tests of one model on one instance per version, with `TESTS_PER_INSTANCE`.

14 Run tests on one instance

1–2 instances

`14_shared_tests.json`

Three tests with the baseline/test pair. 2.5.7 comes from the bucket, and all three 2.5.8 tests run one after another on a **single** instance. OasisLMF is installed and the model pulled only once. The default, `separate`, would launch three instances.

test_1	2.5.7	2.5.8
test_2	2.5.7	2.5.8
test_3	2.5.7	2.5.8

> Each test is still downloaded, compared and published on its own, and a failing test doesn't stop the rest. Caches are cleared before each test so later tests aren't sped up by earlier ones. The instance runs for the sum of the tests' runtimes, so raise `MAX_LIFETIME_HOURS` for long tests. Comparing a shared run against a separate baseline is flagged as "not like for like".

```
{
    "AWS_REGION": "eu-west-1",
    "AMI_ID": "ami-09079da11cd2861fa",
    "INSTANCE_TYPE": "m6i.xlarge",
    "SECURITY_GROUP_ID": "sg-xxxxxxxxxxxxxxxxx",
    "SUBNET_ID": "subnet-xxxxxxxx",
    "IAM_INSTANCE_PROFILE": "YourSSMInstanceProfile",
    "AWS_PROFILE": "oasis",
    "REPO_LOCATIONS": [
        "s3://my-model-bucket"
    ],
    "TESTS": [
        "test_1",
        "test_2",
        "test_3"
    ],
    "OASISLMF_BASELINE_VERSION": "2.5.7",
    "OASISLMF_TEST_VERSION": "2.5.8",
    "BENCHMARK_BUCKET": "s3://my-benchmark-bucket",
    "RESULT_DIRECTORY": "./runs_shared_tests",
    "TESTS_PER_INSTANCE": "shared",
    "MAX_LIFETIME_HOURS": 4
}
```

Laying out a model for TESTS

The model needs one directory per test under `tests/`, each with its own `oasislmf.json`. The tests can share the model's data through relative paths.

```
s3://my-model-bucket/
├── model_data/   keys_data/   meta-data/model_settings.json
└── tests/
    ├── test_1/oasislmf.json   ("model_data_dir": "../../model_data", ...)
    └── test_2/oasislmf.json
```

Every path in a test's `oasislmf.json` is relative to that test's directory, so check each `../../…` path exists in the bucket. A wrong `model_settings_json` stops the run straight away with `Invalid model_settings file or file path`.

Options in example 13

KEY	DEFAULT	USE
`EXECUTION_MODE`	`parallel`	`sequential` runs one at a time, e.g. to stay under an EC2 limit. `DEBUG` (stepping through each command) only works in `sequential`; `parallel` switches it off.
`COMPARISON_TOLERANCE`	`0.005`	Relative tolerance for numeric CSV cells in the output diff (0.5%). Raise it if sampling noise shows up as a FAIL.
`INSTANCE_TYPE`	`t3.medium`	Use a fixed-performance type for timing. These examples use `m6i.xlarge` (4 vCPUs, 16 GB), the type the stored baselines ran on. Burstable `t2` / `t3` types can slow down mid-run when their CPU credits run out, and a run on a different type than its baseline is flagged as not like for like.
`DISK_GB`	`100`	Raise for large model data.
`MAX_LIFETIME_HOURS`	`2`	Written to the instance's `ALPACA_END_TIME` tag, marking when it's overdue. Alpaca terminates every instance itself when its run finishes.
`SSH_MAX_RETRIES`	`60`	Connection attempts, 3 seconds apart, while the instance boots.
`LOG_LEVEL`	`INFO`	`DEBUG` for more detail.

`EC2_NAME` is ignored by a benchmark. Every instance is named `Alpaca {model} {test} {version}` so you can tell them apart in the AWS console. A shared instance takes the name of its first test.

When a run fails

A failed run doesn't stop the others and is never published. Its results folder gets a `failure.txt` with the stage that failed (instance setup, model run or downloading results), the error line that matters, any input file its `oasislmf.json` pointed at that doesn't exist, and the failed command's full output.
The report lists every failed run before the comparisons:

```
Failed runs:
- my-model-bucket test_1 2.5.8: failed during model run
    The path /home/ubuntu/tests/test_1/oasislmf.json (MDK config. JSON file) is indicated as preexisting but does not
exist
    missing input file: tests/test_1/oasislmf.json
    details: ./runs/my-model-bucket-test_1-2.5.8/failure.txt
```

And the last lines in the terminal point straight at the evidence:

```
Benchmark finished with failures: see the report above.
Failure details:
- my-model-bucket test_1 2.5.8: ./runs/my-model-bucket-test_1-2.5.8/failure.txt
Full report: ./runs/benchmark_report.txt
Exiting with status 1.
```

The command also exits with status 1 when outputs differ, so a scheduled or CI benchmark never looks green when it isn't. A run that's only "not like for like" is reported but doesn't fail the command.

Configs rejected before anything launches

1. Setting both `PATH_TO_OASISLMF_JSON` and `TESTS`, or neither.
2. No OasisLMF install at all: no versions, no pair and no branches.
3. Only one of `OASISLMF_BASELINE_VERSION` / `OASISLMF_TEST_VERSION`, both set to the same version, or the pair used with `OASISLMF_VERSIONS`.
4. `PUBLISH_BASELINE` without `BENCHMARK_BUCKET`, or with only branches.
5. A `TESTS` entry that isn't a bare directory name (`test_1`, not `tests/test_1`).
6. An `s3://` `RESULT_DIRECTORY`, since the comparison reads local files.
7. `TESTS_PER_INSTANCE` set to anything other than `separate` or `shared`, or `shared` without `TESTS`.

Matches Alpaca `main` as of 6 October 2026. Every config here passes Alpaca's own config checks. Replace the placeholder AWS values before running.